

Οδηγός YAML Experiments

Τελευταία ενημέρωση: 2026-06-30

Βασική αρχή

Τα feature steps παράγουν raw columns μόνο. Κάθε παράγωγη στήλη, όπως ratio, distance, slope, lag, crossing flag, threshold flag, rolling z-score ή clipping, δηλώνεται ως helper μέσα στο ίδιο feature step.

Το παλιό standalone `feature_transforms` step έχει καταργηθεί. Δεν υπάρχει `tsfresh_rolling.py` helper και το `tsfresh_rolling` απορρίπτεται από το YAML validation.

Trade Path Diagnostics

Τα trade lifecycle diagnostics είναι reporting/artifact layer και δεν μπαίνουν ποτέ στο feature selection. Όταν το top-level `diagnostics.enabled` είναι `true`, το `diagnostics.trade_path.enabled` γίνεται αυτόματα `true`, εκτός αν δοθεί explicit opt-out.

```
diagnostics:
  enabled: true
  # trade_path auto-enabled by default
  trade_path:
    thresholds_r: [0.5, 1.0, 1.5, 2.0]
    include_counterfactuals: true
    plots:
      enabled: true
```

Για explicit opt-out:

```
diagnostics:
  enabled: true
  trade_path:
    enabled: false
```

Τα structured outputs γράφονται κάτω από `report_assets/` όταν υπάρχουν αρκετά trade metadata: `trades_enriched.csv`, `trade_paths.parquet` ή fallback `trade_paths.csv`, `probability_trade_quality.csv`, `counterfactual_exit_summary.csv` και diagnostic JSON warnings. Τα legacy HTML diagnostics όπως `trade_diagnostics_*.html` και `report.html` δεν παράγονται πλέον.

Νέα δομή φακέλων

- `src/features/technical/`: raw indicators όπως `trend`, `atr`, `vwap`, `ppo`
- `src/features/helpers/`: γενικές μετατροπές όπως `ratio.py`, `difference.py`, `lag.py`, `reciprocal.py`, `flags.py`, `rolling_mean.py`, `rolling_std.py`, `rolling_sum.py`, `rolling_linear_regression.py`, `rms.py`, `slope.py`, `rolling_clip.py`, `rolling_zscore.py`

- `src/features/helpers/normalizations/`: trading normalizations όπως `returns.py`, `atr_distances.py`, `volatility.py`, `rolling_zscores.py`, `rolling_percent_rank.py`, `robust_zscore.py`, `volatility_scaled_return.py`, `atr_scaled_distance.py`, `range_position.py`, `realized_vol_percentile.py`, `volume_relative.py`, `rolling_beta_residual.py`
- `src/features/helpers/apply.py`: εφαρμόζει τα helper blocks μετά από κάθε raw feature step
- `src/features/helpers/registry.py`: registry για transform helpers και normalization helpers
- `config/experiments/market_making/`: research-only event-driven market-making experiments, όπως `market_making_moment.yaml`. Τα outputs γράφονται κάτω από `logs/experiments/market_making/` και δεν παράγουν HTML/PPTX artifacts.

Για πλήρη κατηγοριοποιημένη ερμηνεία κάθε helper, δες το `catalog/helpers.md`. Για πλήρη κατηγοριοποιημένη ερμηνεία των raw feature steps, δες το `catalog/features.md`. Για το ίδιο επίπεδο ανάλυσης σε signals, targets και models, δες τα `catalog/signals.md`, `catalog/targets.md` και `catalog/models.md`.

Σειρά εκτέλεσης

Για κάθε entry στο `features`:

1. Εκτελείται το raw feature function από το `step`.
2. Εφαρμόζονται τα `normalizations` του ίδιου step.
3. Εφαρμόζονται τα `transforms` του ίδιου step.
4. Εφαρμόζεται το optional `outputs` mapping.

Σε multi-asset run, τα `params_by_asset` αλλάζουν τα raw feature params. Τα `transforms_by_asset` και `normalizations_by_asset` κάνουν override ανά helper name, αλλά αφήνουν τα υπόλοιπα top-level helpers να ισχύουν.

Συνολική pipeline σειρά

Η πραγματική σειρά στο canonical experiment pipeline είναι:

1. Φόρτωση data και PIT checks.
2. `features`: raw feature steps και οι helper μετατροπές τους.
3. `model` και `model_stages`: target construction για training, fit ανά split και out-of-sample predictions.
4. Top-level `signals`: μετατροπή feature/model outputs σε τελικό trading signal.
5. Optional top-level `target`: μόνο για post-signal diagnostics όταν `model.kind`: `none`.
6. `backtest` ή `portfolio` evaluation.
7. `diagnostics`, `monitoring`, artifacts και optional `execution`.

Δύο σημεία θέλουν προσοχή:

- Αν το target εκπαιδεύει model, μπαίνει μέσα στο `model.target`.
- Αν θέλεις target diagnostics πάνω σε rule-only signal χωρίς model, βάζεις top-level `target`, αλλά τότε το `model.kind` πρέπει να είναι `none`.

Το `model` stage τρέχει πριν από το top-level `signals`. Άρα ένα target μέσα στο `model.target` δεν μπορεί να εξαρτάται από στήλες που παράγονται μόνο από το top-level `signals`. Αν χρειάζεσαι primary candidate πριν από model training, χρησιμοποίησε causal feature step ή compatibility signal step μέσα στο `features`,

όπως `ehlers_semiscalp_long`, `indicator_model_adaptive_pullback`, `roc_long_only_conditions`, `ema_stoch_rsi_pullback` ή `vwap_rms_ema_cross_long`.

Causality και leakage rules

- Τα `features` κοιτάνε μόνο παρελθόν και current bar.
- Τα `normalizations` μέσα στα features πρέπει να είναι causal. Για rolling στατιστικά προτίμησε shifted stats όταν το helper το υποστηρίζει.
- Τα `targets` επιτρέπεται να κοιτάνε μέλλον, αλλά μόνο για label construction.
- Target columns όπως `label`, `target_fwd_*`, `event_ret`, `trade_r`, `oriented_r`, `hit_type` και barrier diagnostics δεν μπαίνουν ποτέ σε `feature_cols` ή signal filters.
- Τα model-driven signals πρέπει να χρησιμοποιούν out-of-sample outputs. Για classifier/regressor σήματα, έλεγξε ότι υπάρχει `pred_is_oos = 1` στα rows που αξιολογείς.
- Τα ML scalars δηλώνονται στο `model.preprocessing` και εφαρμόζονται μέσα στα train folds. Δεν είναι feature normalizations.

Παράδειγμα raw indicator με transforms

```
features:
  - step: trend
    params:
      price_col: close
      sma_windows: []
      ema_spans: [50]
      ema_col_template: ema_50
      add_ratios: false
    transforms:
      ratio:
        enabled: true
        params:
          numerator_col: close
          denominator_col: ema_50
          output_col: close_over_ema_50
          subtract: 1.0
      rms:
        enabled: true
        params:
          source_col: ema_50
          window: 192
          shift: 0
          output_prefix: ema_50
```

Το `trend` παράγει μόνο `ema_50`. Τα `close_over_ema_50` και `ema_50__root_mean_square` παράγονται από helpers.

Παράδειγμα normalizations

```
features:
  - step: atr
    params:
      high_col: high
      low_col: low
      close_col: close
      windows: [14]
      atr_col: atr_14
    normalizations:
      volatility:
        enabled: true
        params:
          close_col: close
          atr_col: atr_14
          add_atr_pct: true
          add_atr_percentile: true
          percentile_window: 252
      rolling_zscores:
        enabled: true
        params:
          columns: [atr_14]
          window: 96
          shift_stats: true
```

Τα normalizations δημιουργούν νέα columns, αλλά δεν είναι ML scalers. Οι ML scalers γίνονται μόνο μέσα στα train folds.

Παράδειγμα per-asset overrides

```
features:
  - step: vwap
    params:
      high_col: high
      low_col: low
      close_col: close
      volume_col: volume
      windows: [20]
      vwap_col: vwap_20
    params_by_asset:
      GER40:
        windows: [32]
        vwap_col: vwap_32
    transforms:
      ratio:
        enabled: true
        params:
          numerator_col: close
          denominator_col: vwap_20
          output_col: close_over_vwap_20
          subtract: 1.0
```

```

transforms_by_asset:
  GER40:
    ratio:
      enabled: true
      params:
        numerator_col: close
        denominator_col: vwap_32
        output_col: close_over_vwap_32
        subtract: 1.0

```

Για τα assets χωρίς override εφαρμόζεται το top-level `ratio`. Για `GER40`, το `ratio` αντικαθίσταται από το asset-specific block.

Διαθέσιμα transform helpers

- `ratio`: παράγει `numerator / denominator - subtract`, με optional `denominator_offset`.
- `difference`: παράγει `source - source.shift(periods)`.
- `lag`: παράγει `source.shift(periods)`.
- `reciprocal`: παράγει `1 / source` ή `1 / abs(source)`.
- `threshold_flag`: παράγει binary flag με `gt`, `ge`, `lt`, `le`, `eq`, `ne`.
- `rising_flag`: παράγει flag όταν `source_t > source_{t-periods}`.
- `between_flag`: παράγει flag όταν η τιμή είναι μέσα σε `[lower, upper]`.
- `crossing_flag`: παράγει cross-up ή cross-down event σε numeric threshold.
- `rolling_mean`: trailing rolling mean.
- `rolling_std`: trailing rolling standard deviation.
- `rolling_sum`: trailing rolling sum.
- `rolling_linear_regression`: trailing slope/intercept/R2.
- `rms`: rolling root mean square.
- `slope`: rolling linear slope helper.
- `rolling_clip`: causal rolling quantile clipping.
- `rolling_zscore`: causal rolling z-score helper.

Κάθε helper δέχεται `enabled`, `params` ή `items`. Το `items` χρησιμοποιείται όταν θέλουμε πολλαπλά outputs από τον ίδιο helper.

Παράδειγμα με `items`:

```

features:
  - step: roofing_filter
    params:
      price_col: close
      output_col: roofing_filter_48_10
    transforms:
      crossing_flag:
        items:
          - source_col: roofing_filter_48_10
            threshold: 0.0
            direction: up
            output_col: roofing_filter_48_10_cross_up

```

```
- source_col: roofing_filter_48_10
  threshold: 0.0
  direction: down
  output_col: roofing_filter_48_10_cross_down
```

Παράδειγμα Hilbert derived columns:

```
features:
- step: hilbert_transform
  params:
    price_col: close
    window: 64
    amplitude_col: hilbert_amplitude_64
    phase_col: hilbert_phase_64
    instantaneous_frequency_col: hilbert_frequency_64
  transforms:
    reciprocal:
      params:
        source_col: hilbert_frequency_64
        use_abs: true
        output_col: hilbert_dominant_cycle_64
    between_flag:
      params:
        source_col: hilbert_dominant_cycle_64
        lower: 10.0
        upper: 48.0
        output_col: hilbert_cycle_ok_64
    rising_flag:
      params:
        source_col: hilbert_amplitude_64
        periods: 3
        output_col: hilbert_amplitude_rising_64
```

Διαθέσιμα normalization helpers

- **returns**: past-looking simple και log returns
- **atr_distances**: ATR-normalized distances ανά ζεύγος columns
- **volatility**: **atr_pct** και rolling ATR percentile
- **rolling_zscores**: z-scores για λίστα columns με optional shifted stats
- **rolling_percent_rank**: percentile rank του current value έναντι prior trailing window.
- **robust_zscore**: rolling median/MAD z-score με shifted stats by default.
- **volatility_scaled_return**: $\text{return} / \text{volatility}$.
- **atr_scaled_distance**: $(\text{base} - \text{reference}) / \text{ATR}$.
- **range_position**: θέση τιμής μέσα στο trailing high-low range.
- **realized_vol_percentile**: percentile rank για realized volatility column.
- **volume_relative**: $\text{volume} / \text{rolling_mean}(\text{volume})$ και optional volume z-score.
- **rolling_beta_residual**: single-factor rolling beta residual έναντι benchmark returns.

Παράδειγμα trading normalizations:

```

features:
- step: returns
  normalizations:
    rolling_percent_rank:
      params:
        source_col: close_logret_1
        window: 252
        output_col: close_logret_1_percent_rank_252
    robust_zscore:
      params:
        source_col: close_logret_1
        window: 252
        output_col: close_logret_1_robust_z_252
    volatility_scaled_return:
      params:
        return_col: close_logret_1
        volatility_col: vol_rolling_96
        output_col: close_logret_1_over_vol_96
    volume_relative:
      params:
        volume_col: volume
        window: 96
        output_col: volume_relative_96

```

Targets

Τα targets απαντούν "τι έγινε μετά από αυτό το timestamp;". Χρησιμοποιούνται είτε για να εκπαιδεύσουν model (`model.target`) είτε ως diagnostics σε rule-only workflow (top-level `target` με `model.kind: none`).

Διαθέσιμα canonical target kinds:

- `forward_return`: fixed-horizon binary label από μελλοντική απόδοση.
- `future_return_regression`: continuous future return label για regressors.
- `triple_barrier`: upper/lower/time barrier label.
- `directional_triple_barrier`: meta-label για ήδη προτεινόμενη πλευρά.
- `r_multiple`: R-multiple outcome για manual long candidates.

Παράδειγμα fixed-horizon classifier target

```

model:
  kind: logistic_regression_clf
  pred_prob_col: pred_prob
  pred_is_oos_col: pred_is_oos
  target:
    kind: forward_return
    price_col: close
    horizon: 12
    threshold: 0.0
    fwd_col: target_fwd_12

```

```

    label_col: label
  split:
    method: purged
    train_size: 24000
    test_size: 4000
    step_size: 4000
    purgeBars: 12
    embargoBars: 12
  feature_cols:
    - close_logret_1
    - atr_over_price_14
    - close_over_ema_50

```

Εδώ το `label = 1` σημαίνει ότι το `close` μετά από 12 bars είναι πάνω από το τρέχον `close`. Το `pred_prob` είναι πιθανότητα positive label, όχι απευθείας εντολή αγοράς.

Παράδειγμα regression target

```

model:
  kind: lightgbm_regressor
  pred_ret_col: pred_ret
  pred_is_oos_col: pred_is_oos
  target:
    kind: future_return_regression
    price_col: close
    horizon: 8
    label_col: label
    raw_fwd_col: raw_fwd_8
    normalize_by_volatility: true
    volatility_col: atr_14
    clip: [-5.0, 5.0]
  split:
    method: purged
    train_size: 30000
    test_size: 5000
    step_size: 5000
    purgeBars: 8
    embargoBars: 8
  feature_selectors:
    include:
      - startswith: [ema_, atr_, ppo_]

```

Το `pred_ret` είναι forecast στην ίδια κλίμακα με το target. Αν το target είναι volatility-normalized, `pred_ret = 1.2` σημαίνει περίπου 1.2 μονάδες τοπικής μεταβλητότητας, όχι 120%.

Παράδειγμα meta-label target με primary candidate

```

features:
  - step: indicator_model_adaptive_pullback

```



```

params:
  direction_col: direction
  signal_col: signal_raw
  candidate_col: signal_candidate
  score_col: signal_score

model:
  kind: lightgbm_clf
  outputs:
    pred_prob_col: pred_prob
    pred_is_oos_col: pred_is_oos
  target:
    kind: directional_triple_barrier
    outputs:
      label_col: label
      event_ret_col: dtb_event_ret
      r_col: dtb_event_r
      oriented_r_col: dtb_oriented_r
    direction_col: direction
    candidate_col: signal_candidate
    price_col: close
    open_col: open
    high_col: high
    low_col: low
    volatility_col: atr_14
    entry_price_mode: next_open
    profit_barrier_r: 1.4
    stop_barrier_r: 1.0
    vertical_barrierBars: 6
    neutral_label: stop
    add_r_multiple: true
  feature_selectors:
    include:
      - startswith: [ema_, atr_, rsi_, stoch_]
      - exact: [direction, signal_score]

```

Η σειρά είναι: το feature-compatible primary step γράφει **direction** και **signal_candidate**, το **model.target** φτιάχνει labels μόνο στα candidates, και το model μαθαίνει πιθανότητα επιτυχίας του candidate. Το **direction** μπορεί να είναι feature γιατί είναι causal output του primary rule. Το **label** και τα **dtb_*** outputs δεν πρέπει να είναι features.

Παράδειγμα rule-only diagnostic target

```

model:
  kind: none

signals:
  kind: ppo_adx_stochrsi_trend
  params:
    signal_col: signal
    position_col: position

```

```

    entry_long_col: entry_long
    exit_long_col: exit_long

target:
  kind: triple_barrier
  label_col: tb_label
  fwd_col: tb_event_ret
  r_col: tb_event_r
  oriented_r_col: tb_oriented_r
  price_col: close
  open_col: open
  high_col: high
  low_col: low
  volatility_col: atr_over_price
  label_mode: meta
  side_col: signal
  candidate_mode: side_change
  entry_price_mode: next_open
  max_holding: 96
  upper_mult: 2.5
  lower_mult: 1.5
  add_r_multiple: true

```

Αυτό δεν εκπαιδεύει model. Χρησιμεύει για να δεις αν τα rule-only entries έχουν καλό barrier outcome και R distribution.

Signals

Τα signals μετατρέπουν features, model probabilities ή forecasts σε τελικό `signal_col`. Το `backtest.signal_col` πρέπει να δείχνει στη στήλη που θέλεις να αξιολογήσεις.

Συνηθισμένες κατηγορίες:

- Rule/indicator signals: `trend_state`, `rsi`, `momentum`, `stochastic`, `volatility_regime`.
- Primary candidate generators: `orb_candidate_side`, `roc_long_only_conditions`, `ema_stoch_rsi_pullback`, `indicator_model_adaptive_pullback`, `ppo_adx_stochrsi_trend`, `stc_roofing_hilbert`.
- Model probability signals: `probability_threshold`, `probability_conviction`, `probability_vol_adjusted`, `meta_probability_side`, `manual_long_model_filter`.
- Forecast signals: `dense_return_forecast`, `forecast_threshold`, `forecast_vol_adjusted`.
- Wrappers/filters: `regime_filtered`.

Παράδειγμα flat/EDA signal

```

signals:
  kind: none
  params:
    signal_col: eda_flat_signal

```

```
backtest:
  signal_col: eda_flat_signal
```

Χρήσιμο όταν θέλεις να τρέξει όλο το reporting χωρίς πραγματικές θέσεις.

Παράδειγμα rule signal

```
signals:
  kind: trend_state
  params:
    state_col: trend_regime
    signal_col: signal_trend_state
    mode: long_short_hold

backtest:
  engine: vectorized
  signal_col: signal_trend_state
  returns_col: close_ret
```

Το `trend_state` διαβάζει ήδη υπάρχον causal feature column και γράφει τελικό long/short/flat signal.

Παράδειγμα meta probability filter

```
signals:
  kind: meta_probability_side
  outputs:
    signal_col: signal
  params:
    prob_col: pred_prob
    side_col: direction
    candidate_col: signal_candidate
    threshold: 0.58
    profit_barrier_r: 1.4
    stop_barrier_r: 1.0
    min_expected_value_r: 0.1
    mode: long_short

backtest:
  engine: manual_barrier
  signal_col: signal
```

Το signal κρατά την πλευρά του primary candidate μόνο αν το OOS `pred_prob` περάσει threshold και, αν έχει δοθεί, expected-value φίλτρο. Δεν αντιστρέφει πλευρά από μόνο του.

Παράδειγμα long-only model filter

```

signals:
  kind: manual_long_model_filter
  params:
    prob_col: pred_prob
    candidate_col: signal_candidate
    base_signal_col: signal_side
    threshold: 0.55
    signal_col: model_filtered_long_signal

backtest:
  engine: manual_barrier
  signal_col: model_filtered_long_signal
  long_only: true

```

Κατάλληλο όταν το primary setup είναι long-only και το model απλώς αποφασίζει αν το trade αξίζει να κρατηθεί.

Παράδειγμα forecast threshold signal

```

signals:
  kind: forecast_threshold
  params:
    forecast_col: pred_ret
    signal_col: signal_forecast
    upper: 0.002
    lower: -0.002
    mode: long_short_hold

backtest:
  engine: vectorized
  signal_col: signal_forecast

```

Χρησιμοποιείται μετά από regressor/forecaster. Τα thresholds πρέπει να είναι στην ίδια κλίμακα με το `pred_ret`.

Models

Το `model.kind` επιλέγει τον trainer ή policy builder. Τα πιο συνηθισμένα outputs είναι:

- `pred_prob`: πιθανότητα positive label ή candidate success.
- `pred_ret`: continuous return forecast.
- `pred_vol`: volatility/risk forecast.
- `pred_is_oos`: ένδειξη ότι το prediction είναι out-of-sample.
- `signal_rl` και `action_rl`: policy outputs από RL models.

Διαθέσιμα model groups:

- Classifiers: `logistic_regression_clf`, `elastic_net_clf`, `lightgbm_clf`, `xgboost_clf`, `event_transformer_encoder`.
- Regressors/forecasters: `lightgbm_regressor`, `sarimax_forecaster`, `garch_forecaster`, `lstm_forecaster`, `patchtst_forecaster`, `tft_forecaster`, `chronos_bolt_forecaster`, `chronos_2_forecaster`, `timesfm_2p5_200m_forecaster`, `timesfm_1p0_200m_forecaster`.
- Feature discovery: `tsfresh_extrema_feature_discovery`.
- Single-asset RL: `ppo_agent`, `dqn_agent`.
- Portfolio RL: `ppo_portfolio_agent`, `dqn_portfolio_agent`.

Παράδειγμα classifier model

```
model:
  kind: xgboost_clf
  pred_prob_col: pred_prob
  pred_is_oos_col: pred_is_oos
  preprocessing:
    scaler: robust
  target:
    kind: forward_return
    price_col: close
    horizon: 8
    threshold: 0.001
    label_col: label
  split:
    method: purged
    train_size: 30000
    test_size: 5000
    step_size: 5000
    expanding: true
    purgeBars: 8
    embargoBars: 8
  feature_cols:
    - close_logret_1
    - close_over_ema_50
    - atr_over_price_14
  params:
    n_estimators: 300
    max_depth: 3
    learning_rate: 0.03
    random_state: 7
```

Το model παράγει `pred_prob`. Για trading, το μετατρέπεις σε signal με `probability_threshold`, `probability_conviction`, `meta_probability_side` ή άλλο probability signal.

Παράδειγμα sequence forecaster

```
model:
  kind: lstm_forecaster
  pred_ret_col: pred_ret
```

```

pred_is_oos_col: pred_is_oos
target:
  kind: future_return_regression
  price_col: close
  horizon: 12
  label_col: label
  normalize_by_volatility: true
  volatility_col: atr_14
split:
  method: purged
  train_size: 36000
  test_size: 6000
  step_size: 6000
  purge_bars: 12
  embargo_bars: 12
feature_cols:
  - close_logret_1
  - atr_over_price_14
  - ppo_hist
  - stoch_rsi_k
params:
  lookback: 64
  hidden_size: 64
  epochs: 20
  batch_size: 256
  scale_target: true

```

Sequence models χρειάζονται αυστηρό split και αρκετά δεδομένα. Μην συγκρίνεις training confidence με OOS performance.

Παράδειγμα staged model/overlay

```

model:
  kind: xgboost_clf
  pred_prob_col: pred_prob_tree
  pred_is_oos_col: pred_is_oos
  target:
    kind: directional_triple_barrier
    direction_col: direction
    candidate_col: signal_candidate
    price_col: close
    open_col: open
    high_col: high
    low_col: low
    volatility_col: atr_14
    label_col: label
  feature_selectors:
    include:
      - startswith: [ema_, atr_, rsi_]

model_stages:

```

```
- kind: tft_forecaster
  enabled: true
  pred_ret_col: pred_ret_tft
  target:
    kind: future_return_regression
    price_col: close
    horizon: 8
    label_col: tft_label
  feature_cols:
    - close_logret_1
    - pred_prob_tree
    - atr_over_price_14
```

Χρησιμοποίησε staged models μόνο όταν ξέρεις ποια outputs παράγει κάθε stage και ότι τα downstream stages βλέπουν μόνο OOS-safe columns.

Reinforcement learning

Τα RL models δεν εκπαιδεύουν classifier probability. Μαθαίνουν policy που γράφει actions ή άμεσο signal. Συνήθως:

- `action_rl`: raw action ή action id.
- `signal_rl`: trading exposure που θα χρησιμοποιήσει το backtest.
- `pred_is_oos`: OOS policy evaluation flag.

Για RL runs, το top-level `signals` συνήθως μένει `none`, επειδή το model παράγει ήδη `signal_rl`. Το `backtest.signal_col` πρέπει να δείχνει στο `signal_rl`.

Παράδειγμα single-asset PPO

```
model:
  kind: ppo_agent
  signal_col: signal_rl
  action_col: action_rl
  pred_is_oos_col: pred_is_oos
  target:
    kind: forward_return
    price_col: close
    horizon: 1
  split:
    method: purged
    train_size: 30000
    test_size: 5000
    step_size: 5000
    expanding: true
    purgeBars: 1
    embargoBars: 1
  backtest:
    returns_col: close_ret
    returns_type: simple
  feature_cols:
```

```

- close_ret
- atr_over_price_14
- close_over_ema_50
- ppo_hist
env:
  window_size: 32
  execution_lagBars: 1
  action_space: continuous
  max_signal_abs: 1.0
  reward:
    cost_per_turnover: 0.00015
    slippage_per_turnover: 0.0001
    inventory_penalty: 0.00002
    switching_penalty: 0.00005
  params:
    total_timesteps: 50000
    learning_rate: 0.0003
    gamma: 0.99
    device: cpu

signals:
  kind: none
  params: {}

backtest:
  engine: vectorized
  signal_col: signal_rl
  returns_col: close_ret

```

To `signal_rl = 0.75` σημαίνει 75% long exposure στην κλίμακα του environment. Δεν είναι πιθανότητα 75%.

Παράδειγμα single-asset DQN

```

model:
  kind: dqn_agent
  signal_col: signal_rl
  action_col: action_rl
  pred_is_oos_col: pred_is_oos
  target:
    kind: forward_return
    price_col: close
    horizon: 1
  split:
    method: purged
    train_size: 30000
    test_size: 5000
    step_size: 5000
    purge_bars: 1
    embargo_bars: 1
  feature_cols:

```



```

- close_ret
- atr_over_price_14
- rsi_14
env:
  window_size: 32
  execution_lagBars: 1
  action_space: discrete
  max_signal_abs: 1.0
  reward:
    cost_per_turnover: 0.00015
    slippage_per_turnover: 0.0001
    switching_penalty: 0.00005
  params:
    total_timesteps: 25000
    learning_starts: 1000
    buffer_size: 100000
    batch_size: 256
    train_freq: 4
    gradient_steps: 1
    target_update_interval: 1000
    learning_rate: 0.0005
    gamma: 0.99
    device: cpu

signals:
  kind: none
  params: {}

backtest:
  engine: vectorized
  signal_col: signal_rl

```

Το `action_rl` είναι action id. Διάβασέ το μέσω του action mapping του environment, όχι σαν αριθμητικό score.

Παράδειγμα portfolio DQN

```

model:
  kind: dqn_portfolio_agent
  signal_col: signal_rl
  action_col: action_rl
  pred_is_oos_col: pred_is_oos
  data_alignment: inner
  target:
    kind: forward_return
    price_col: close
    horizon: 1
  split:
    method: purged
    train_size: 36000
    test_size: 6000

```

```
    step_size: 6000
    expanding: true
    max_folds: 4
    purgeBars: 1
    embargoBars: 1
  backtest:
    returns_col: close_ret
    returns_type: simple
  feature_cols:
    - close_ret
    - ehlers_roofing_atr
    - ehlers_hilbert_amplitude_z_252
    - atr_over_price_14
  env:
    window_size: 32
    execution_lag_bars: 1
    action_space: discrete
    max_signal_abs: 1.0
    reward:
      cost_per_turnover: 0.00015
      slippage_per_turnover: 0.0001
      inventory_penalty: 0.00002
      switching_penalty: 0.00005
  params:
    total_timesteps: 25000
    learning_starts: 1000
    buffer_size: 100000
    batch_size: 256
    train_freq: 4
    gradient_steps: 1
    target_update_interval: 1000
    learning_rate: 0.0005
    gamma: 0.99
    device: cpu

signals:
  kind: none
  params: {}

backtest:
  engine: vectorized
  signal_col: signal_r1
  returns_col: close_ret

portfolio:
  enabled: true
  construction: signal_weights
  long_short: true
  gross_target: 1.0
  constraints:
    min_weight: -0.5
    max_weight: 0.5
```

```
max_gross_leverage: 1.0
target_net_exposure: 0.0
```

Σε portfolio RL, το `signal_rl` κάθε asset είναι μέρος ενιαίας portfolio action. Μην αξιολογείς κάθε asset σαν ανεξάρτητο strategy χωρίς να κοιτάς gross/net exposure, turnover και constraints.

Πότε προτιμάς RL

- Όταν η απόφαση είναι sequential policy/action και όχι απλή πρόβλεψη label.
- Όταν θέλεις το reward να περιλαμβάνει turnover, costs, inventory penalty ή portfolio constraints.
- Όταν έχεις αρκετά δεδομένα για σταθερό OOS policy evaluation.

Αν δεν έχεις καθαρό edge σε supervised labels, το RL σπάνια θα το δημιουργήσει μαγικά. Ξεκίνα από rule baseline ή supervised meta-labeling και πήγαινε σε RL μόνο όταν θες να μάθεις policy πάνω σε ήδη χρήσιμα state features.

Robust scaler στα models

Στα classifier models υποστηρίζονται πλέον:

```
model:
  kind: logistic_regression_clf
  preprocessing:
    scaler: robust
```